Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU Assignment 8: GPUs

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: `to37gulo`
Student HPC-Account: `ptfs356v`

Gehan Younes
Student IdM: `qi39dyry`

Ahmed Mohamed
Student IdM: `gi10dafy`

July 1, 2026

1. Theoretical Peak Performance (16 Credits)

The hypothetical cluster specifications are summarised below. An FMA instruction contributes 2 FLOP (one multiply and one add).

CPU

The CPU has 144 cores, each executing 1 AVX-512 FMA per cycle at 2.8 GHz. AVX-512 holds 8 double-precision (DP) or 16 single-precision (SP) values.

$$P_{\text{CPU}}^{\text{DP}} = 144 \times 8 \times 2 \times 2.8\,\text{GHz} = \mathbf{6.45}\,\text{TFLOP/s}$$
$$P_{\text{CPU}}^{\text{SP}} = 144 \times 16 \times 2 \times 2.8\,\text{GHz} = \mathbf{12.90}\,\text{TFLOP/s}$$

Single GPU

Each GPU has 92 SMs × 4 SMSPs = 368 SMSPs, each executing 1 DP FMA or 16 SP FMAs per cycle at 1.2 GHz.

$$P_{\text{GPU}}^{\text{DP}} = 368 \times 1 \times 2 \times 1.2\,\text{GHz} = \mathbf{0.883}\,\text{TFLOP/s}$$
$$P_{\text{GPU}}^{\text{SP}} = 368 \times 16 \times 2 \times 1.2\,\text{GHz} = \mathbf{14.13}\,\text{TFLOP/s}$$

All GPUs of a Single Node (8 GPUs)

$$P_{\text{node}}^{\text{DP}} = 8 \times 0.883 = \mathbf{7.07}\,\text{TFLOP/s} \qquad P_{\text{node}}^{\text{SP}} = 8 \times 14.13 = \mathbf{113.05}\,\text{TFLOP/s}$$

All GPUs in the Cluster (212 nodes × 8 GPUs = 1696 GPUs)

$$P_{\text{cluster}}^{\text{DP}} = 1696 \times 0.883 = \mathbf{1.498}\,\text{PFLOP/s} \qquad P_{\text{cluster}}^{\text{SP}} = 1696 \times 14.13 = \mathbf{23.97}\,\text{PFLOP/s}$$

Unit	DP [TFLOP/s]	SP [TFLOP/s]
CPU (1 node)	6.45	12.90
Single GPU	0.883	14.13
Node (8 GPUs)	7.07	113.05
Cluster (1696 GPUs)	1497.9	23966.5

Table 1: Theoretical peak floating-point performance.

2. Dense Matrix-Matrix Multiplication (16 Credits)

For two square matrices of size $N = 4096$, the number of floating-point operations is $2N^3 = 2 \times 4096^3 \approx 137.44\,\text{GFLOP}$. The total PCIe bandwidth is $4 \times 32 = 128\,\text{GB/s}$ per direction.

(a) CPU Only

$$t_{\text{CPU}}^{\text{DP}} = \frac{137.44\,\text{GFLOP}}{6451.2\,\text{GFLOP/s}} \approx \mathbf{21.3}\,\text{ms} \qquad t_{\text{CPU}}^{\text{SP}} = \frac{137.44\,\text{GFLOP}}{12902.4\,\text{GFLOP/s}} \approx \mathbf{10.7}\,\text{ms}$$

(b) GPUs Only — Matrices Already on Device

$$t_{\text{GPU}}^{\text{DP}} = \frac{137.44\,\text{GFLOP}}{7065.6\,\text{GFLOP/s}} \approx \mathbf{19.5}\,\text{ms} \qquad t_{\text{GPU}}^{\text{SP}} = \frac{137.44\,\text{GFLOP}}{113049.6\,\text{GFLOP/s}} \approx \mathbf{1.22}\,\text{ms}$$

(c) GPUs — With Host-Device Transfers (No Overlap)

Data to transfer: 2 input matrices host→device, 1 output matrix device→host.

Double precision (8 B/element):

$$t_{\text{H2D}}^{\text{DP}} = \frac{2 \times 4096^2 \times 8\,\text{B}}{128\,\text{GB/s}} \approx 2.10\,\text{ms} \qquad t_{\text{D2H}}^{\text{DP}} = \frac{4096^2 \times 8\,\text{B}}{128\,\text{GB/s}} \approx 1.05\,\text{ms}$$

$$t_{\text{total}}^{\text{DP}} = 19.5 + 2.10 + 1.05 \approx \mathbf{22.6}\,\text{ms}$$

Single precision (4 B/element):

$$t_{\text{H2D}}^{\text{SP}} \approx 1.05\,\text{ms} \qquad t_{\text{D2H}}^{\text{SP}} \approx 0.52\,\text{ms}$$

$$t_{\text{total}}^{\text{SP}} = 1.22 + 1.05 + 0.52 \approx \mathbf{2.79}\,\text{ms}$$

Speedup of Scenario (c) over Scenario (a)

$$S^{\text{DP}} = \frac{21.3\,\text{ms}}{22.6\,\text{ms}} \approx \mathbf{0.94}\times \quad \text{(GPU with transfers is } \textit{slower} \text{ than CPU for DP!)}$$

$$S^{\text{SP}} = \frac{10.7\,\text{ms}}{2.79\,\text{ms}} \approx \mathbf{3.83}\times$$

The DP result is notable: the hypothetical GPU has only a modest DP advantage over the CPU per node (7.07 vs 6.45 TFLOP/s), so the PCIe transfer overhead tips the balance in favour of the CPU. For SP the GPU is ∼8.8× faster in compute, more than compensating for the transfer overhead.

3. CUDA Kernel Analysis (8 Credits)

The kernel under analysis is:

```
__global__ void kernel(float* A, float* B, float* C, int size) {
  int tidx = threadIdx.x + blockDim.x * blockIdx.x;
  int tidy = threadIdx.y + blockDim.y * blockIdx.y;
  if (tidx >= size || tidy >= size) return;
  C[tidx + tidy*size] += A[tidx + tidy*size] * B[tidy];
}
```

Listing 1: Kernel under analysis

(a) Minimum Threads to Saturate the DRAM Interface

Using Little's Law, the number of outstanding memory requests needed to keep the DRAM interface busy is:

$$N_{\text{requests}} = \frac{BW}{\text{clock}} \times L = \frac{1200\,\text{GB/s}}{1.2\,\text{GHz}} \times 720\,\text{cy} = 1000\,\frac{\text{B}}{\text{cy}} \times 720\,\text{cy} = 720{,}000\,\text{B}$$

With a cache line size of 128 B, this corresponds to:

$$\frac{720{,}000\,\text{B}}{128\,\text{B/CL}} = 5{,}625 \text{ outstanding cache lines}$$

Each warp of 32 threads performing a coalesced access to `float` arrays ($32 \times 4\,\text{B} = 128\,\text{B}$) issues exactly one cache line request. Therefore:

$$\boxed{N_{\text{threads}} = 5{,}625 \times 32 = \mathbf{180{,}000}\ \text{threads}}$$

(b) Thread Block Size to Minimise L2 Cache Traffic

The key observation is that `B[tidy]` depends only on `tidy`, not on `tidx`. Therefore, all threads in a block that share the same `tidy` value access the same element of `B`.

Optimal choice: `blockDim.y = 1`.

With `blockDim.y = 1`, every thread in a block has the same `tidy`, so `B[tidy]` is loaded from L2 exactly *once* per block and reused by all `blockDim.x` threads. If `blockDim.y` $>$ `1`, there are multiple distinct `tidy` values within the block, requiring separate L2 loads for each.

`blockDim.x` should be a multiple of 32 (warp size) to ensure that accesses to `A` and `C` at `[tidx + tidy*size]` are coalesced (consecutive in the `x`-direction).

Recommended block size: `(blockDim.x = 32 or 64 or 128, blockDim.y = 1)`.

4. CUDA Benchmark: Array Scaling (30 Credits)

Implementation

We implemented a CUDA benchmark that scales all elements of a double-precision array by 2 (`a[i] = 2.0 * a[i]`) using a grid-stride loop. The application accepts four command-line arguments: problem size N, number of repetitions, number of blocks, and block size.

```
__global__ void scale_kernel(double* a, long n) {
    long tid    = threadIdx.x + blockDim.x * (long)blockIdx.x;
    long stride = blockDim.x  * (long)gridDim.x;
    for (long i = tid; i < n; i += stride)
        a[i] = 2.0 * a[i];
}
```

Listing 2: Grid-stride scaling kernel

Timing is performed using CUDA events. The device is synchronised once before and once after all kernel repetitions to ensure accurate measurement. The effective kernel bandwidth is computed as:

$$\text{BW}_{\text{kernel}} = \frac{2 \times N \times 8\,\text{B} \times \text{reps}}{t_{\text{kernel}}}$$

(factor of 2 for one read and one write per element per repetition).

Compilation and Execution

```
# Compile on Alex cluster
module load cuda
nvcc -O3 -o scale scale.cu

# Example: 32M elements, 64 reps, 168 blocks, 256 threads
./scale 33554432 64 168 256
```

The code was run on a single NVIDIA A40 GPU (48 GB GDDR6, 696 GB/s peak DRAM bandwidth) on the Alex cluster at NHR@FAU.

5. Block Size Experiment (12 Credits)

Setup

- Problem size: $N = 2^{25} = 33{,}554{,}432$ elements (256 MiB)
- Repetitions: 64
- Number of blocks: 168
- Block sizes tested: 32, 64, 96, 128, 192, 256, 320, 384, 512, 640, 768, 1024

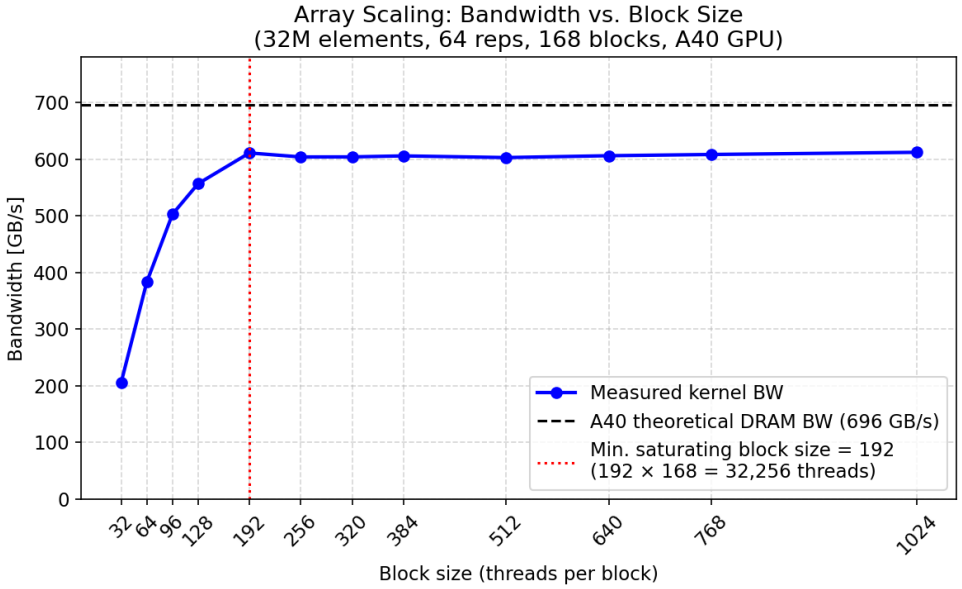


Figure 1: Effective kernel bandwidth vs. block size on the A40 GPU (32M elements, 64 repetitions, 168 blocks).

Results and Analysis

Block size	BW [GB/s]
32	205.3
64	383.0
96	502.7
128	556.7
192	610.9
256	603.9
320	604.0
384	605.7
512	602.8
640	606.0
768	608.2
1024	612.0

Table 2: Bandwidth vs. block size (32M elements, 64 reps, 168 blocks).

The bandwidth rises steeply from 32 to 192 threads per block, then flattens out. The minimum block size that saturates the memory bandwidth is **192 threads/block**.

At this configuration, the total number of active threads is:

$$192 \text{ threads/block} \times 168 \text{ blocks} = \mathbf{32{,}256} \text{ threads}$$

Note that the 180,000-thread estimate from Question 3(a) applies to the *hypothetical* GPU (1200 GB/s bandwidth), not the A40 (696 GB/s). The A40 analogue would be $\approx$ 104,000 threads. The grid-stride loop explains why 32,256 threads still suffice: each thread issues many independent memory requests in sequence, keeping the DRAM pipeline full and effectively hiding latency.

The peak measured bandwidth is ≈ 612 GB/s, which is $612/696 \approx \mathbf{88}\%$ of the A40's theoretical DRAM bandwidth.

6. Problem Size Experiment (18 Credits)

Setup

- Grid: 1344 blocks $\times$ 256 threads = 344,064 total threads
- Repetitions: 32
- Problem sizes: $N = 1$ to 10^9 elements
- GPU: NVIDIA A40 on Alex cluster

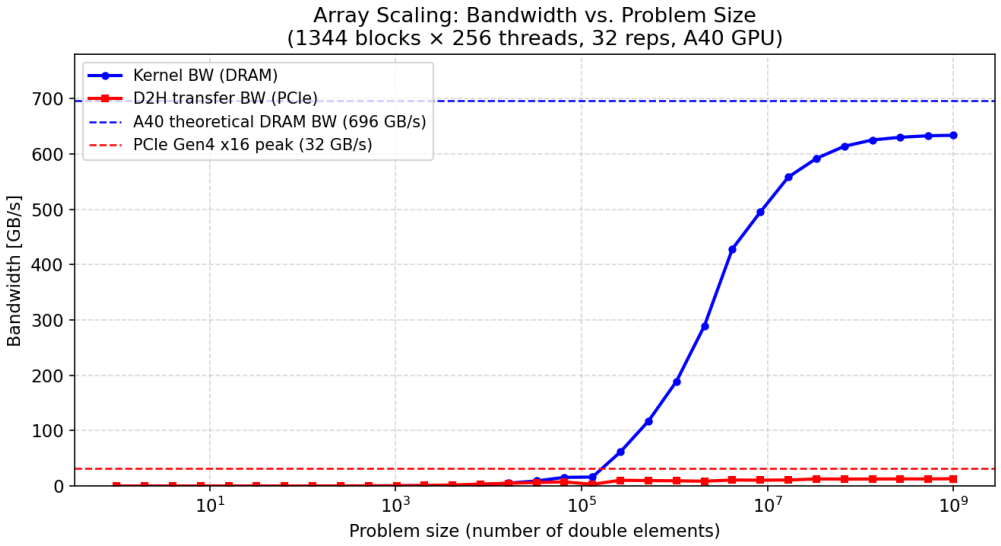


Figure 2: Effective kernel bandwidth (DRAM) and D2H transfer bandwidth vs. problem size on the A40 GPU.

Kernel Bandwidth vs. Problem Size

For small N, the bandwidth is very low because the total working set fits in the GPU's L2 cache (6 MB on the A40), and/or there are insufficient threads in flight to hide the DRAM latency. The transition from cache-dominated to DRAM-dominated behaviour occurs around $N \approx 750{,}000$ elements, where the unique data ($N \times 8\,\text{B} \approx 6\,\text{MB}$) equals the A40's L2 cache size. This is consistent with the characteristic S-curve shape observed in the lecture's stream benchmark.

For large N the bandwidth saturates at approximately $\mathbf{634\,GB/s}$, which is $634/696 \approx \mathbf{91}\%$ of the A40's theoretical DRAM bandwidth of 696 GB/s.

D2H Transfer Bandwidth vs. Problem Size

The device-to-host transfer bandwidth saturates at approximately **13 GB/s** for large arrays, well below the PCIe Gen4 ×16 theoretical peak of 32 GB/s. This is because host memory was allocated with `malloc()`, which produces *pageable* memory. The CUDA runtime must stage pageable transfers through an internal pinned buffer, roughly halving the effective PCIe bandwidth compared to what `cudaMallocHost()` (pinned/page-locked memory) would achieve. The low bandwidth for small N reflects timing resolution: a tiny transfer completes in microseconds, making the measured time noisy.

Summary

Metric	Measured	Theoretical peak
Kernel DRAM BW (large N)	634 GB/s	696 GB/s (91%)
D2H transfer BW (large N)	13 GB/s	32 GB/s (41%)

Table 3: Measured vs. theoretical peak bandwidths on the A40 GPU.

The gap between measured and theoretical DRAM bandwidth is due to real-world memory controller efficiency and ECC overhead. The large gap in PCIe bandwidth is due to the use of pageable host memory.